

Lab 3 Report – Performance Analysis of Part 2

In this report, I aim to compare the time spent servicing interrupts and the time spent running user code. We can achieve this by putting breakpoints to separate each section of code we want to analyze:

- A) Servicing interrupts: I used one breakpoint at the label *SERVICE_IRQ* to indicate the start of the section, and one breakpoint at the label *EXIT_IRQ* to indicate the end of the code section.
- B) Polling slider switches: I used one breakpoint at the instruction *read_slider_switches_ASM* (which is at the beginning of the *IDLE* loop) to indicate the start of the code section, and one breakpoint at the *service_PBs* label (which is when we stop servicing the switches) to indicate the end of the section.
- C) In IDLE or user code (excluding polling of slider switches): I used one breakpoint at the label *service_PBs* (which is where we start the user code after polling the slider switches) to indicate the beginning of the code section, and one breakpoint at the *IDLE* label (which starts with the *read_slider_switches_ASM* instruction), which is the instruction right after we finish the user code (i.e. branch back to the beginning of the loop), to indicate the end of the code section.

Thus, using the Counter pane, we can read the number of instructions at the beginning and end of the code section. The number of instructions will then me the difference. We can also estimate the time spent running a section of code in a similar way using the breakpoints, by subtracting the simulator runtime after the code section was executed with the simulator runtime before. These results are shown in Table 1.

	Servicing interrupts	Polling slider switches		In IDLE or user code			
		No user input	Toggle switch to ACE	No user input	Switch direction to right	Speed up	Pause
# of executed instructions at start	96	121	154	103	145	204	285
# of executed instructions at end	118	129	453	111	170	251	314
# of executed instructions	22	8	299	8	25	47	29
Simulator runtime at start (ms)	10	18	26	4	19	44	59
Simulator runtime at end (ms)	14	22	36	8	26	49	66
Estimated time spent (ms)	4	4	10	4	7	5	7

Table 1. Data of the performance analysis for code sections in Part 2.

We notice from this data that when there is no user input to react to, servicing interrupts takes the most time (based on the number of executed instructions), which makes sense since it must check multiple interrupts and modify the interrupt flags in memory. The polling and IDLE execution time are very short as it only needs to read the slider state or the flags in memory and skips over all the code to update things if no change is needed. However, when there is a user input requiring an update (both from the switches or from the pushbuttons), the time spent executing code is significantly higher, especially for the switch (which could be due to inefficient update of the HEX drivers). Similarly, this can be explained by the fact that many more instructions must be executed when a change is required, which would have been conditionally skipped if there were no input. In conclusion, the time spent servicing interrupts is around two times longer than polling or user code when there is no user input. When there is user input, the time spent executing the user code will be significantly longer and will depend on what is required to be done in each situation.